

实习转正答辩——李雅扉

1 个人简介

就读时间	学历层次	院校	专业
2020-2024	本科	南京理工大学	软件工程
2024-2027	硕士	东南大学	软件工程

2 工作内容

【智能白板：该类型的内容不支持下载】

2.1 批量推理

批量推理适用于需要处理大量数据且无需实时获取结果的场景，例如日志分析、离线数据预测和评测等。与强调低延迟和实时可用性的在线推理不同，批量推理允许请求在完成窗口内排队、分片和重试，关注**任务吞吐**及**资源利用率**。

随着任务规模和模型数量增长，系统面临三个核心问题：

核心问题	我的解决方案
请求负载和资源持续变化，静态并发配置难以兼顾吞吐与稳定性	模型并发自动探测机制
底层资源扩缩，资源暂时不可用时任务失败	资源感知的的任务暂停与恢复机制
输入与结果文件规模增大，传统全量处理方式引发 OOM	大文件请求任务执行链路优化

当前已达成如下效果：

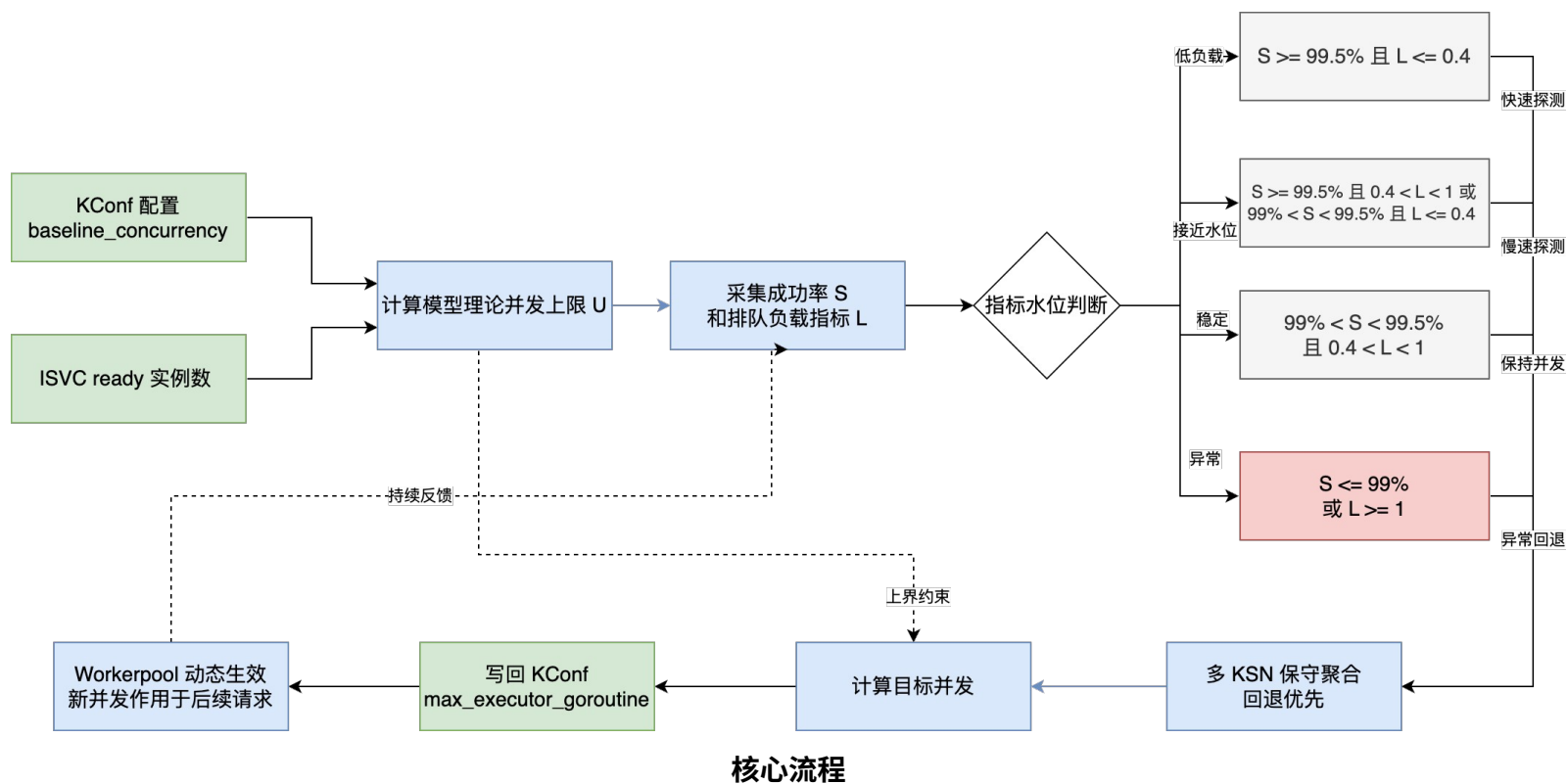
- 开发模型并发自动探测机制，开启自动探测后，KV cache 利用率从 25% 提升至 70%。
- 实现资源感知的任务暂停与恢复，彻底解决因资源缩容、模型实例异常导致的任务失败问题。
- 优化大文件任务执行链路，支持批量推理任务规模从 5G 扩展至 100G 以上大文件。

以下将分别阐述技术细节。

2.1.1 模型并发自动探测机制

背景：批量推理场景中，`max_execute_goroutine` 决定单个模型的并发请求上限。配置过低会造成底层推理资源利用不充分，限制任务吞吐；配置过高则会增加模型服务的运行压力，导致请求失败率上升，影响服务稳定性，核心在于 KV Cache 占用和请求成功率都和模型的实际承载的并发量相关。

底层技术原理：大模型推理过程中，每个运行中的请求都需要缓存历史 Token 的 Key/Value，并在 Decode 阶段随生成 Token 持续增长。因此，并发请求越多，KV Cache 占用通常越高，长 Prompt 和长输出还会进一步放大缓存压力。当模型接近容量上限时，新请求开始进入 waiting queue；持续过载可能引发超时、OOM 等错误，最终表现为 `waiting/running` 升高和请求成功率下降。因此，安全并发不仅取决于副本数，还与实时请求特征和引擎负载有关。



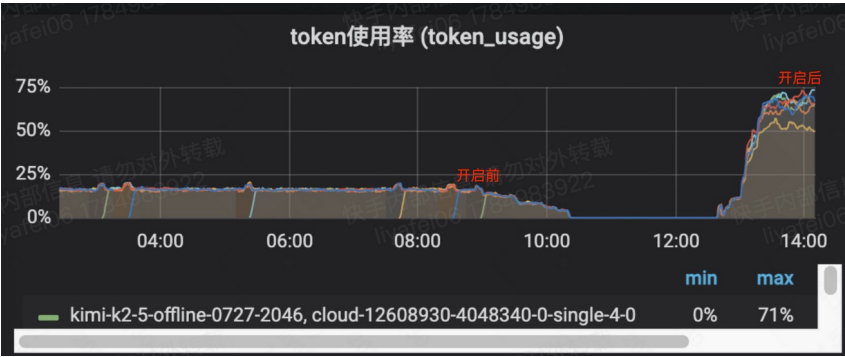
分析了以上原理后，我设计的系统首先根据 KConf 中的单副本基准并发和 ISVC Ready 副本数计算理论并发上限 U ；再持续采集以下两个核心指标：

- 请求成功率 S ：反映网关到模型服务整条链路的实际执行结果；
- 排队负载 $L = \text{waiting queue} / \text{running queue}$ ：反映请求进入速度与模型实际处理能力之间的关系。

根据指标所处水位，动态调整模型并发：

- 快速探测：当 $S > 99.5\%$ 且 $L < 0.4$ 时，成功率健康且队列负载较低，按理论上限 U 的 20% 探测并发；
- 慢速探测：当成功率或队列负载逐渐接近安全水位时，按 U 的 10% 小步增加并发，降低探测过程中的过载风险；
- 保持并发：当指标处于稳定区间时不再增加并发，使系统维持在当前工作点；
- 自动回退：当 $S < 99\%$ 或 $L > 1$ 时，说明请求已经出现明显失败或积压，按 U 的 10% 降低并发。

通过“资源容量上界 + 运行态指标反馈”的闭环控制，系统能够持续逼近模型在当前资源和请求分布下的最大安全并发，在保障服务稳定性的同时提升批量任务吞吐和推理资源利用率。

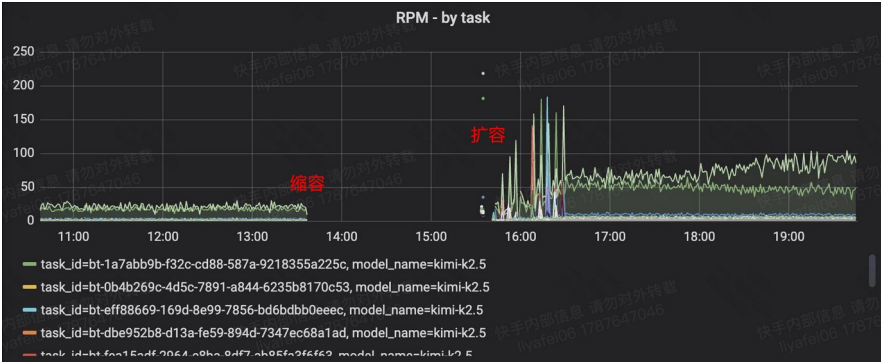


2.1.2 资源感知的任务暂停与恢复机制

背景：批量推理执行过程中，模型服务可能因底层资源缩容而暂时无可用副本。此类请求本质上是“资源暂不可用”，不应被判定为执行失败。

设计并实现资源感知的任务暂停与恢复机制：

- 在 Shard 提交及请求重试前实时检查模型 KSN Ready 副本；
- 当模型实例缩容至 0 副本或暂时不可用时，Shard / 请求在执行点挂起等待；
- 资源恢复后自动继续执行，避免无资源期间请求集中失败、任务重跑或数据丢失，实现弹性扩缩容过程中的任务无感恢复。



注：对于已发送至模型实例的运行中请求，若执行期间因资源缩容而失败，系统会进入重试流程，并在重试前检查模型 Ready 副本数；资源不可用时挂起当前任务，待资源恢复后继续执行，避免将短暂的资源波动判定为任务失败。

2.1.3 大文件请求任务执行链路优化

背景：随着批量推理任务规模增长，原有链路在输入分片阶段容易受网络中断影响，结果合并阶段则存在内存占用过高甚至 OOM 的问题，影响大任务处理的稳定性。

(1) 输入分片链路优化：重构大文件下载与分片流程，采用 HTTP 流式下载和逐行解析，按固定 max_shard_size 持续生成并上传分片到 OSS，将内存占用控制在单个分片范围内，彻底避免 OOM 风险；补充连接超时和流中断重试，避免异常中断产生不完整任务数据。

(2) 大规模任务结果流式合并：基于 S3 Multipart Upload 顺序读取并分段上传 Shard 结果，取代全量下载后在内存聚合的低效设计；补充分段上传重试，降低大文件合并过程中的内存 OOM 和网络异常风险。

2.2 万擎成本智能 agent

万擎资源运营涉及资源、容量和成本的综合决策问题，现有运营数据分散在资源看板、成本看板、天问及 KConf 中。面对容量评估、扩容规划和资源回收等问题，通常需要人工跨系统查询、核对模型与时间口径，完成多轮计算，容易出现以下问题：

核心问题	解决方案
资源、容量和成本数据分散，单一指标无法形成完整结论	汇总分散的运营数据，通过实现单 Agent 多领域路由机制，避免 agent 发散回答
面对不同业务场景，如何构建可复用、可扩展的 Agent 通识能力	构建“Skill 场景路由—Reference 按需加载—Workflow 确定性执行”的分层能力链路

围绕以上问题，已建设万擎成本智能 Agent，将分散的运营指标组织成一条可执行、可追溯的决策链，实现从人工查数向智能决策的升级。

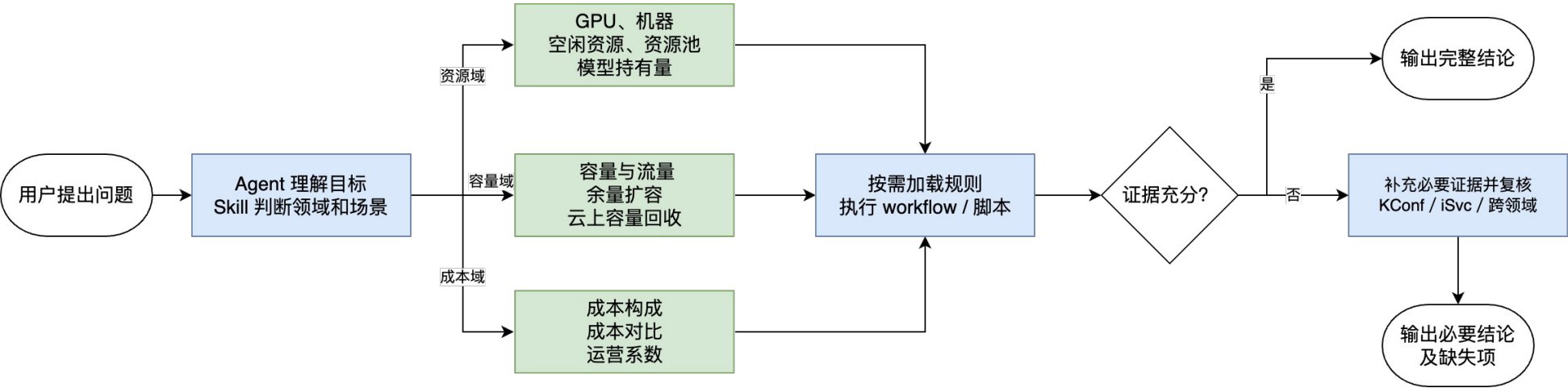
2.2.1 单 Agent 多领域路由机制

背景：万擎运营问题横跨资源、容量和成本领域，相同概念在不同场景中的含义并不一致。例如，“机器数查询”既可能表示物理库存，也可能表示容量缺口换算出的理论需求。简单依靠关键词选择工具，容易出现领域误判和指标混算。

设计并实现单 Agent 多领域路由机制，Agent 能力划分为资源、容量和成本三个领域：

- 资源域负责查询 GPU、机器、资源池、空闲资源、实际可用资源和模型持有量；
- 容量域负责配置容量、流量余量、扩容实例数和云上容量回收场景；
- 成本域负责查询云上成本构成、自部署运营系数及容量回收的成本影响。

Agent 根据用户最终需要得到的指标选择主领域，其他领域只提供形成结论所必需的辅助数据；按场景加载对应的领域 Reference，避免一次性加载全部知识导致不同指标口径相互干扰。



2.2.2 Agent 通识能力建设

背景：Agent 不仅需要理解自然语言，还需要完成场景识别、模型与时间参数提取、任务规划、工具调用和结果组织。如果所有逻辑都依赖提示词或针对验收 Case 逐条实现，复杂场景下容易出现执行步骤不稳定、领域知识难以复用、新增能力扩展成本高等问题，并且难以处理已有业务场景中不同表述方式、参数组合和分析深度的泛化问题。

构建基于 Skill、Reference 和 Workflow 的 Agent 通识能力体系：

- (1) 通过 Skill 定义场景分类、能力边界和执行入口，根据用户最终目标进行场景路由，使不同自然语言表述能够归一到明确的业务场景；
- (2) 通过 Reference 按需加载指标口径、领域知识和分析方法，使业务规则与 Agent 主提示词解耦，能够独立维护并持续扩展；
- (3) 设计参数化 Workflow 和脚本，接收模型、provider、时间范围、卡型和分析深度等结构化参数，固化数据查询、参数校验和计算过程，使 Agent 能够稳定调用 CK、KConf MCP 等数据接口；

（4）将模型解析、时间窗口处理、任务编排、批量执行、失败隔离和结果组织沉淀为公共能力，供资源、容量和成本场景复用。

在通识能力基础上，进一步落地容量余量与扩容分析、云上容量回收与成本影响评估等典型场景。在已有领域场景和指标范围内，Agent 可以处理不同问题表述、参数组合及分析深度，实现面向现有业务场景的泛化。

3 收获成长

实习期间，我围绕批量推理和万擎成本智能 Agent 两条主线，参与了从问题分析、方案设计、开发实现到验证上线的完整过程，有以下收获：

1. 系统设计与工程能力

通过批量推理项目，深入理解了大规模异步任务从数据接入、任务调度、模型执行到结果交付的全生命周期链路，在具体功能开发中实践了 Kafka、Redis、ClickHouse 等，了解了各中间件在异步消息传递、任务状态管理和指标分析等场景中的作用，提升了对复杂系统整体架构和模块协作关系的理解。

2. 业务抽象与 Agent 建设能力

通过开发万擎成本智能 Agent，将分散的业务知识、指标口径和人工操作流程，抽象为可复用的 Skill 与 Workflow，具备了较好大模型 Agent 的开发能力。

3. VibeCoding 协同开发能力

在开发过程中实践 Vibe Coding，使用 AI 辅助代码检索和方案实现，极大提升了开发与迭代效率；同时由自己负责需求拆解、架构设计、关键决策和异常处理策略，并通过代码 Review 和灰度验证把控实现质量，在提升研发效率的同时保证代码质量与可控性。

4. 问题闭环与责任意识

形成“发现问题—及时回滚—分析原因—迭代优化—上线灰度验证”的闭环思路，在保障系统稳定性的基础上持续推进方案落地。

4 未来规划

未来希望继续深耕 MaaS 领域，深入参与从模型优化、资源调度、业务落地的完整链路，致力于大模型场景下对 GPU 资源的极致管理。后续，希望在异构 GPU 算力的性能提升，以及极致的弹性扩缩容等前沿领域做功，提高 GPU 利用率、推理吞吐与整体资源效率。